

CSE 816: Software Production Engineering

DETAILED FINAL PROJECT REPORT

**RedBus: Microservices-Based Ticketing Platform
with DevOps Automation**

Project Domain: Transport Ticketing / DevOps Automation
Department: Department of Computer Science
Submission Date: May 12, 2026

Submitted By

Name	Roll Number
Rahul Raman	MT2025100
Affan Shaikh	MT2025016

GitHub Repository: Rahul09123/Red-Bus

Contents

1	Abstract	1
2	Introduction	1
2.1	Problem Context	1
2.2	Project Goals	1
3	System Architecture	3
3.1	High-Level Architecture Diagram	3
3.2	Service-by-Service Responsibilities	3
3.2.1	API Gateway	3
3.2.2	Eureka Server	4
3.2.3	Member Service	4
3.2.4	Security Service	4
3.2.5	Expedition Service	4
3.2.6	Payment Service	4
4	Service Communication Model	5
4.1	Request Flow	5
4.2	Operational Data Flow	5
5	Frontend Architecture	6
6	CI/CD and Automation	7
6.1	Jenkins Pipeline Architecture	7
6.2	Build and Packaging Strategy	7
6.3	Deployment Automation	8
6.3.1	Service Build Pipelines	9
7	Kubernetes Orchestration	11
7.1	ConfigMap Usage (k8s/configmap.yaml)	12
7.2	Secret Usage (k8s/deprecated-secrets/secrets.yaml)	12

7.3	API Gateway Role in Kubernetes	12
7.4	Ingress Configuration (k8s/ingress.yaml)	13
7.5	Health and Self-Healing	13
7.6	Horizontal Scaling with HPA (k8s/hpa.yaml)	13
7.7	ELK on Kubernetes (k8s/elk-stack.yaml)	13
8	Configuration Management with Ansible	14
9	Observability and Monitoring (ELK Stack)	14
10	Security and Secret Management	15
11	Results, Evaluation, and Conclusion	17
11.1	Evaluation Alignment	17
11.2	Conclusion	18

1 Abstract

RedBus is implemented as a production-oriented bus ticketing platform where software functionality and software delivery are developed together as one engineering system. The project demonstrates how a modern ticketing application can be split into independent microservices while still being deployed and operated through a single DevOps pipeline. Instead of focusing only on user-facing features, this implementation gives equal importance to automation, observability, scaling behavior, and operational resilience.

The platform includes role-based workflows, containerized services, Jenkins-driven CI/CD, Kubernetes orchestration, and ELK-based centralized logging. The result is a complete software production engineering artifact in which code changes move through build, packaging, deployment, and runtime monitoring with minimal manual intervention.

2 Introduction

2.1 Problem Context

A ticketing system naturally contains multiple business domains: identity, account management, route/expedition search, booking, and payment. In a monolithic setup, these domains become tightly coupled, making releases riskier and scaling less efficient. The project addresses this by splitting responsibilities into separate services and introducing automated operational workflows for reliable deployments.

2.2 Project Goals

The target was not just “build an app,” but build an app that can be delivered repeatedly and safely. Therefore, the project goals included:

1. Domain-wise microservice separation with clear boundaries.
2. Fully automated build and deployment pipeline.
3. Kubernetes-native deployment model with probes and autoscaling.
4. Centralized logs for distributed-system troubleshooting.
5. Security-aware handling of runtime credentials and secrets.

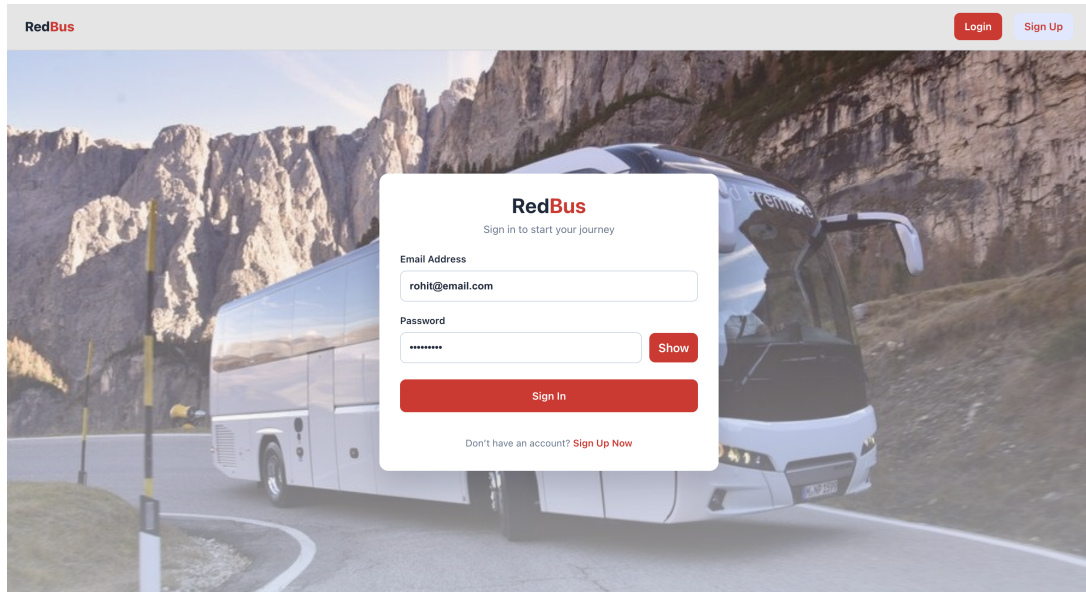


Figure 1: Initial frontend state of the RedBus platform

Explanation: This interface snapshot shows the user-facing entry point and supports the narrative that the platform is organized around role-aware user journeys rather than a single generic dashboard.

3 System Architecture

3.1 High-Level Architecture Diagram

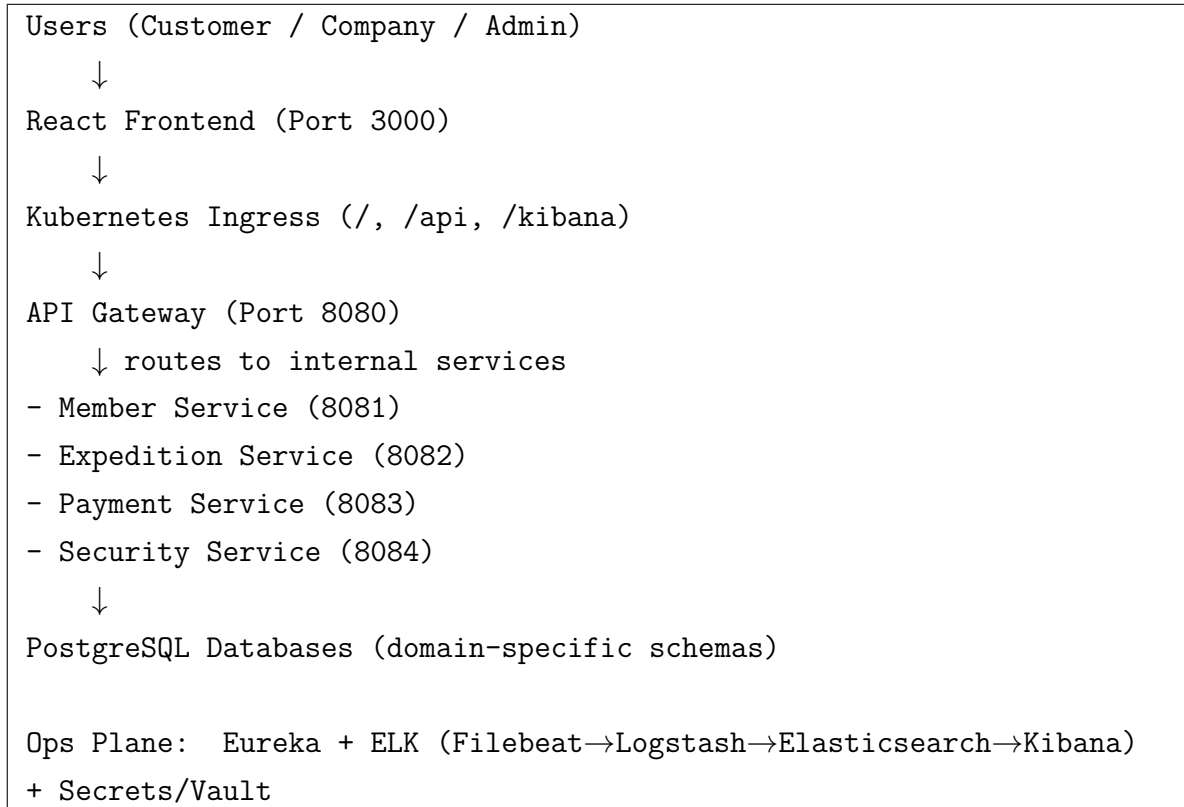


Figure 2: Logical architecture diagram of the RedBus platform

Explanation: All client requests flow through a controlled path (Ingress to Gateway to services), which centralizes policy and avoids exposing internal microservices directly. This architecture also separates business traffic from observability traffic, improving maintainability and debugging.

3.2 Service-by-Service Responsibilities

3.2.1 API Gateway

The API Gateway is the unified entry point for backend APIs. It receives requests from the frontend and forwards them to member, expedition, payment, or security services based on route patterns. This design simplifies frontend integration and supports centralized error handling and security checks.

3.2.2 Eureka Server

Eureka acts as a service discovery registry for Spring Cloud services. It supports dynamic registration and lookup, which is useful in distributed deployments where service instances can restart or scale.

3.2.3 Member Service

The member service manages user and profile-related operations such as registration flows and account-level details. It isolates identity/profile domain logic from booking and payment concerns.

3.2.4 Security Service

The security service handles session lifecycle operations, including session creation, validation, and logout for different user roles (customer/company/admin). Keeping security logic centralized avoids duplication and inconsistent auth behavior across services.

3.2.5 Expedition Service

This service owns the transport domain: expeditions, route listing, seat visibility, and ticket-related workflows. It is core to customer and company operational flows and can be scaled independently when booking/search load increases.

3.2.6 Payment Service

The payment service handles payment-domain operations and transaction status management. It is decoupled to preserve transactional concerns independently from expedition logic.

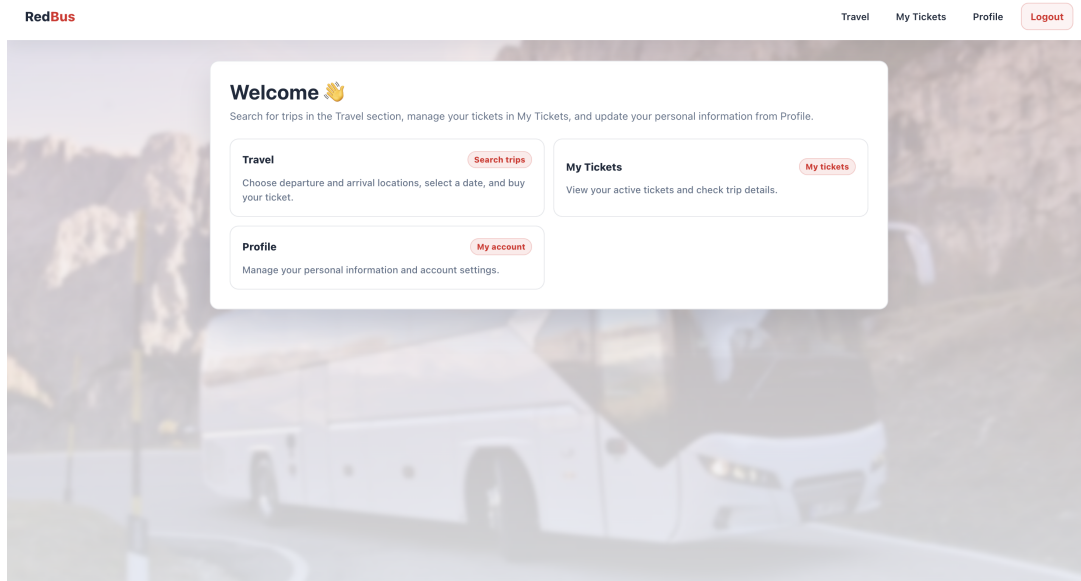


Figure 3: Role-driven platform screen

Explanation: This image supports the service-boundary discussion by showing that user interactions are already segmented into contextual flows, which map cleanly onto backend domain services.

4 Service Communication Model

4.1 Request Flow

The platform uses synchronous HTTP/REST communication over Kubernetes internal networking for business operations:

1. User interaction starts in the React frontend.
2. Ingress receives and routes the request.
3. API Gateway resolves the target operation and forwards to the correct microservice.
4. Target microservice processes business logic and accesses domain data.
5. Response is returned to frontend through the same gateway path.

4.2 Operational Data Flow

In addition to business calls, the system has an observability flow:

1. Pods generate runtime logs.
2. Filebeat collects logs at node level.
3. Logstash parses and enriches logs.
4. Elasticsearch stores logs for search.
5. Kibana provides dashboard-level analysis.

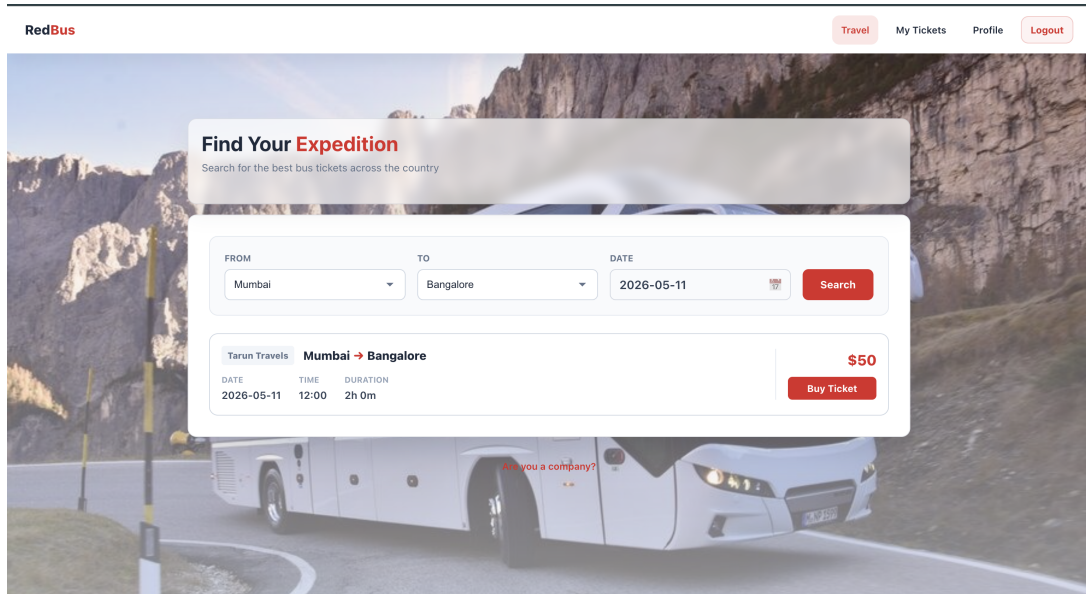


Figure 4: Functional user flow evidence

Explanation: This screenshot demonstrates one stage of the operational user journey, aligning with the gateway-mediated communication path described above.

5 Frontend Architecture

The frontend is built using React and Vite with route-level guards and role-specific layouts. Authentication routes are separated from protected routes, and customer/company/admin journeys are isolated through dedicated route groups. This improves maintainability by keeping user-role behavior explicit in routing and layout composition.

From an SPE perspective, this frontend structure reduces accidental privilege crossover and keeps business workflow transitions clear for testing and debugging.

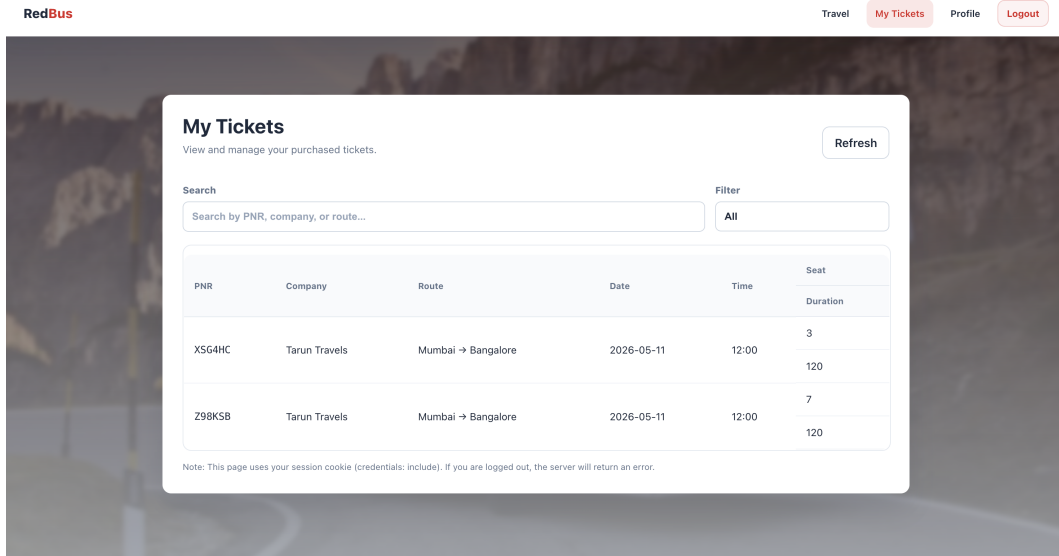


Figure 5: Additional frontend state in role-aware navigation

Explanation: This figure further shows that UI transitions are contextual and governed by the authenticated role and selected user action.

6 CI/CD and Automation

6.1 Jenkins Pipeline Architecture

The CI/CD flow is built around a Jenkins orchestrator pipeline at repository root and dedicated child pipelines for each service (frontend, api-gateway, eureka-server, member-service, security-service, expedition-service, and payment-service). The root pipeline is triggered by `githubPush()`, so every commit to `main` starts an automated release cycle.

In the **Build & Push Services** stage, Jenkins triggers all child jobs in parallel. This is important for microservices because build latency scales with the slowest component; parallelization keeps the complete cycle short and avoids a long serial queue.

6.2 Build and Packaging Strategy

Each child pipeline compiles and packages its code using service-appropriate tooling (Maven for Spring services, npm/Vite for frontend), then builds Docker images and pushes them to Docker Hub. The pipelines rely on Jenkins credential bindings for registry login and never hardcode credentials in pipeline logic.

At orchestrator level, Jenkins computes a commit short-hash and replaces `:latest` image tags in Kubernetes manifests with that hash before deployment. This gives immutable

deployment traceability: every running pod can be mapped back to a specific Git commit.

6.3 Deployment Automation

The deployment stage applies Kubernetes manifests in a deterministic sequence:

1. Namespace and ConfigMap are applied first.
2. Docker registry secret is recreated from Jenkins credentials.
3. Application and ELK secrets are applied.
4. Core infrastructure (Vault, Postgres, pgAdmin, Eureka) is deployed.
5. Business services and frontend are deployed.
6. Ingress, ELK resources, and HPA resources are applied.

This ordered flow ensures dependencies are ready before dependent services start. The final verification stage uses `kubectl get pods` and `kubectl get svc` in the target namespace to confirm rollout status.

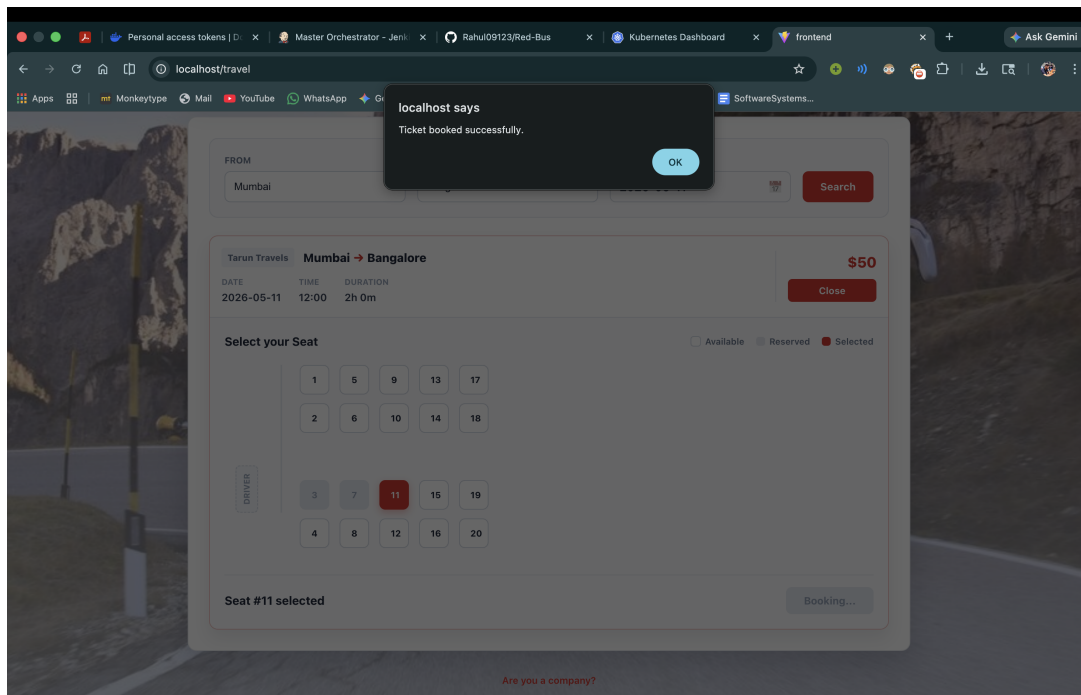


Figure 6: Pipeline or deployment transition evidence

Explanation: This screenshot is used to validate the CI/CD narrative that code and deployment actions are executed through structured automation stages.



Figure 7: Master Orchestrator Pipeline

Explanation: The master orchestrator is the root Jenkins pipeline that coordinates the entire CI/CD workflow. It triggers all child service pipelines in parallel, manages the build orchestration, and handles the deployment sequencing to Kubernetes. This orchestrator ensures that code changes from GitHub trigger a complete automated release cycle, with commit hashes being used for immutable image versioning and traceability. The orchestrator verifies all services have been built successfully before proceeding to deployment, and it ensures proper ordering of resource creation in Kubernetes to respect dependencies between services.

6.3.1 Service Build Pipelines

Each microservice has its own dedicated Jenkins child pipeline that handles service-specific build logic:

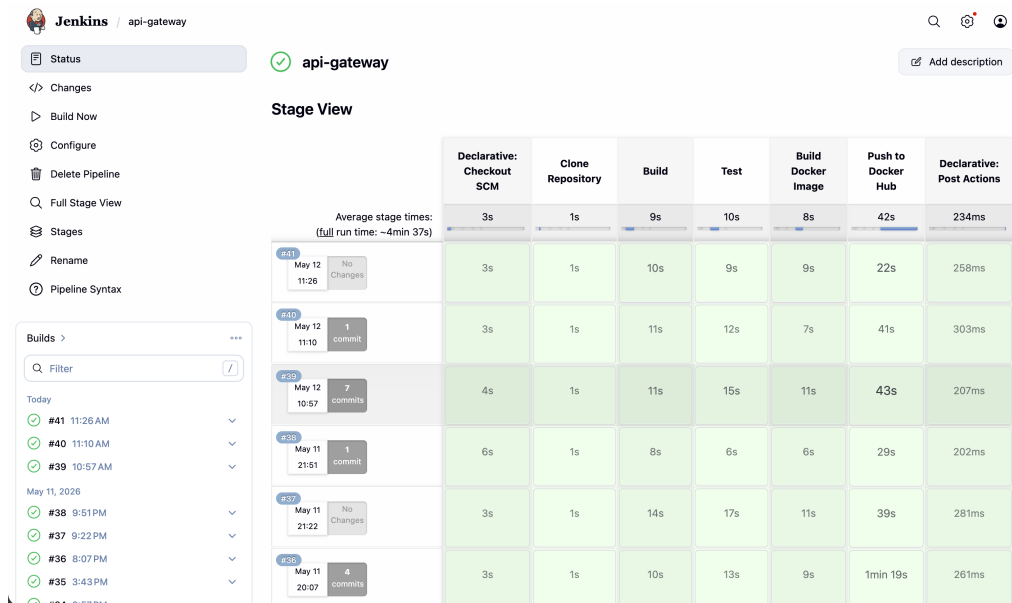


Figure 8: API Gateway Service CI/CD Pipeline

Explanation: The API Gateway service pipeline demonstrates the complete build flow for a critical microservice. It includes stages for code compilation, unit testing, Docker image building, and image registry push. The pipeline uses Maven for Java-based Spring Cloud Gateway compilation and generates Docker images tagged with the commit hash for immutable deployment references. The API Gateway is central to routing all external requests to appropriate backend services, making its reliable build and deployment crucial to system stability.

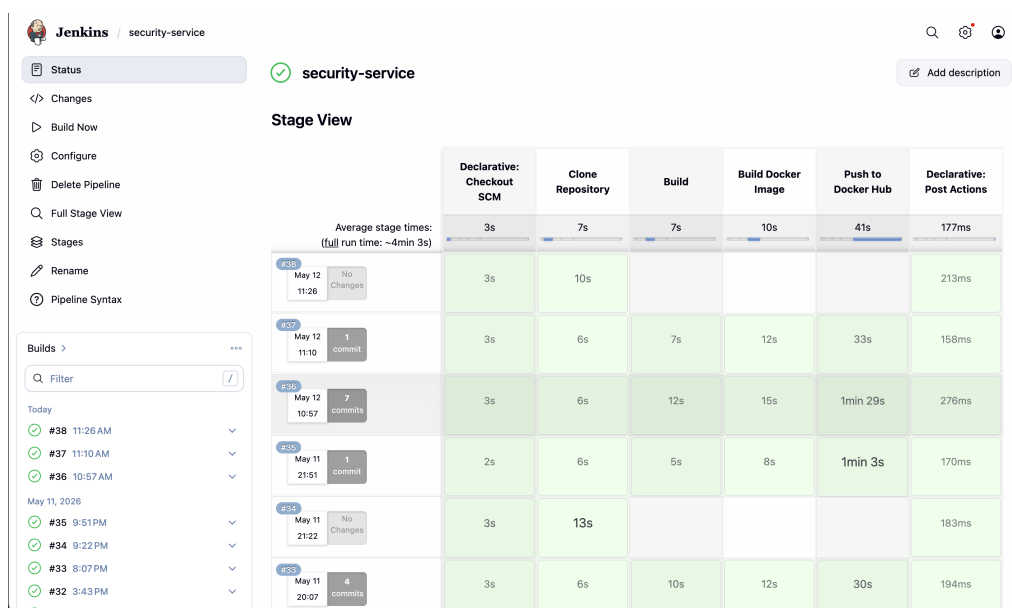


Figure 9: Security Service CI/CD Pipeline

Explanation: The Security Service pipeline manages the authentication and authoriza-

tion microservice. This service is responsible for JWT token generation, user credential validation, and session management. The pipeline ensures that security-related code changes are built with the same rigor as other services, with automated testing and containerization. The immutable tagging approach ensures that security patches can be traced back to specific Git commits, providing an audit trail for compliance and security investigations.

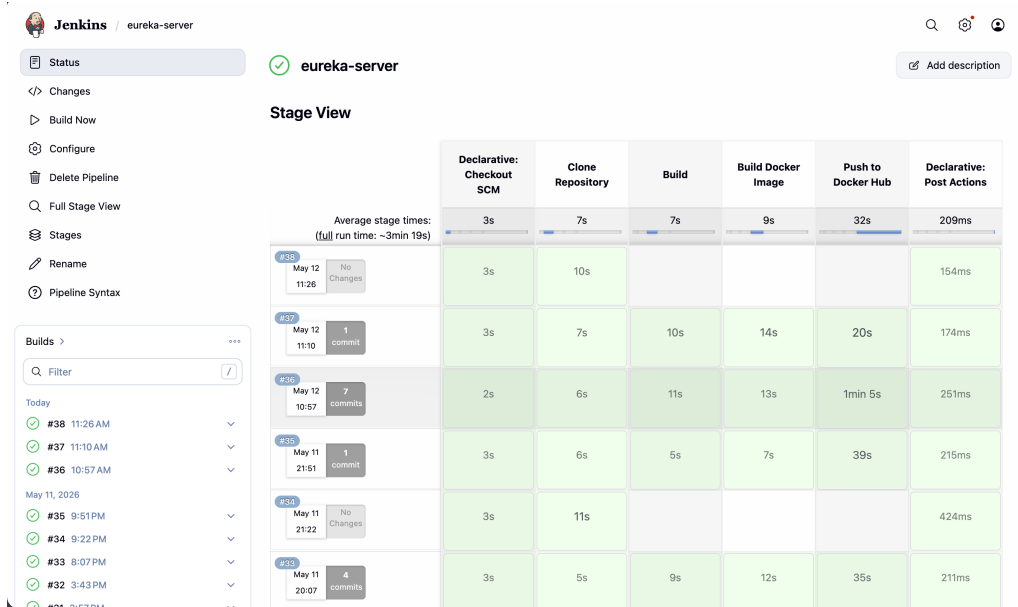


Figure 10: Eureka Service Registry CI/CD Pipeline

Explanation: The Eureka service registry pipeline builds the service discovery component that enables all microservices to locate each other dynamically. This pipeline creates a containerized Eureka server that maintains the real-time registry of service instances and their health status. All other microservices depend on Eureka for bootstrapping, making its reliable deployment a prerequisite for system functionality. The pipeline ensures consistent tagging and versioning that aligns with the overall orchestration strategy.

7 Kubernetes Orchestration

Kubernetes is the primary runtime control plane for the project. Every major component is represented as declarative YAML under `k8s/`, which makes infrastructure versioned and repeatable. The platform runs in namespace `redbus` with independent Deployments and Services for gateway, business microservices, frontend, data, and observability stack.

7.1 ConfigMap Usage (`k8s/configmap.yaml`)

The ConfigMap `redbus-config` stores non-secret runtime configuration such as:

- service discovery endpoint (`EUREKA_SERVER`),
- PostgreSQL host/port and username,
- internal service URLs (`member/security/expedition/payment`),
- gateway and frontend base URLs.

Services consume these values using `configMapKeyRef` in environment variables. For example, `member-service` reads `POSTGRES_USER` and `EUREKA_SERVER` from ConfigMap. This separates deploy-time configuration from container image build and avoids baking environment values into code artifacts.

7.2 Secret Usage (`k8s/deprecated-secrets/secrets.yaml`)

Sensitive values are stored in Kubernetes Secrets:

- `redbus-secrets` for database and admin-related sensitive values,
- `docker-credentials` for private image pulling,
- `elk-credentials` for ELK credentials.

In service manifests, sensitive entries are mounted through `secretKeyRef` (e.g., `member-service` reads `POSTGRES_PASSWORD` from `redbus-secrets`). This pattern prevents plain-text passwords in deployment specs and supports secret rotation without rebuilding images.

7.3 API Gateway Role in Kubernetes

The API Gateway deployment is the central backend entry point. It is exposed internally as a ClusterIP service on port 9090 that targets container port 8080. The gateway runs with startup/readiness/liveness probes against `/actuator/health`, ensuring traffic is sent only when the application is healthy.

Because all external API calls pass through this one service, cross-cutting behavior (routing, auth checks, and response normalization) can be centralized. In operational terms, this also simplifies debugging because request failures can be traced starting at one ingress point.

7.4 Ingress Configuration (k8s/ingress.yaml)

Ingress uses class `nginx` and defines `host+path` rules for both `redbus.local` and `localhost`:

- `/api` routes to `api-gateway:9090`,
- `/kibana` routes to `kibana:5601`,
- `/` routes to frontend service.

This gives a clean public interface where frontend, API, and observability UI are accessible through one ingress layer. It also preserves internal service isolation because `member/security/expedition/payment` are not directly exposed outside the cluster.

7.5 Health and Self-Healing

Startup, readiness, and liveness probes are configured so that traffic is sent only to healthy pods and failed pods can be automatically restarted.

7.6 Horizontal Scaling with HPA (k8s/hpa.yaml)

Horizontal Pod Autoscalers are defined for: `api-gateway`, `member-service`, `expedition-service`, `security-service`, `payment-service`, and `frontend`.

Each HPA uses:

- `minReplicas: 1`,
- `maxReplicas: 2`,
- CPU target `averageUtilization: 80`.

This means pods scale out when CPU pressure crosses the target threshold and scale in when load drops. Even with conservative limits, this demonstrates production-style elasticity and removes manual scaling from day-to-day operations.

7.7 ELK on Kubernetes (k8s/elk-stack.yaml)

The ELK stack is fully deployed in Kubernetes:

- Elasticsearch deployment and service for indexed storage.

- Logstash deployment listening on Beats port 5044, with parsing pipeline and Elasticsearch output.
- Kibana deployment exposed on port 5601.
- Filebeat DaemonSet running on nodes to collect pod logs from `/var/log/pods`.

Filebeat uses a ConfigMap-driven config and Kubernetes metadata enrichment, then forwards logs to Logstash. Logstash transforms and ships to Elasticsearch, and Kibana reads from Elasticsearch for querying dashboards. This gives centralized, searchable logs across all microservices and infrastructure pods.

8 Configuration Management with Ansible

Ansible is used for infrastructure automation in addition to Jenkins-based CI/CD. The repository contains playbooks that apply role-based deployment logic such as cluster preparation, namespace setup, secret provisioning, database setup, microservice deployment, monitoring setup, and cleanup.

Two playbook styles are present in the repo:

- `ansible/playbook.yml` for RedBus-focused deployment flow with tags like `vault`, `deploy`, `elk`, and `ingress`.
- `ansible/deploy-ticketing-system.yml` for modular environment deployment and cleanup using role tags (`setup`, `database`, `services`, `monitoring`, `cleanup`).

Operationally, Ansible gives repeatability for environment creation and reduces manual `kubectl` command sequences. This is valuable when setting up fresh machines or recreating environments consistently for testing or demonstration.

9 Observability and Monitoring (ELK Stack)

The observability pipeline uses Filebeat, Logstash, Elasticsearch, and Kibana to support end-to-end operational visibility in the microservices environment. Logs are centralized, enriched with runtime metadata, indexed for search, and visualized for troubleshooting.

In this implementation, Filebeat runs as a DaemonSet and mounts node log paths, which ensures log collection coverage regardless of which node a pod runs on. The Filebeat service account and cluster role permissions allow it to read Kubernetes resource metadata, and that metadata is included in logs for richer filtering in Kibana.

Logstash ingests Beats traffic and applies path-based parsing before writing documents to Elasticsearch indexes. Kibana is made available through Ingress (`/kibana`), so observability access follows the same ingress control plane as application traffic. This architecture improves incident response quality because logs from multiple services can be correlated in one place by namespace, pod, and container.

10 Security and Secret Management

Security concerns are addressed through dedicated auth/session management in the security service and secret handling through Kubernetes secrets and Vault-related assets. This separates sensitive runtime material from application logic and supports safer deployment practices.

At runtime:

- Non-sensitive values are loaded via ConfigMap.
- Sensitive values are injected via Secrets.
- Docker pull credentials are provided through `imagePullSecrets`.

This split is important because configuration updates (for endpoints/ports) and secret rotation (for passwords/tokens) follow different security and operational lifecycles.

For production hardening, placeholder credentials should be replaced by environment-specific rotated secrets and strict access policies.

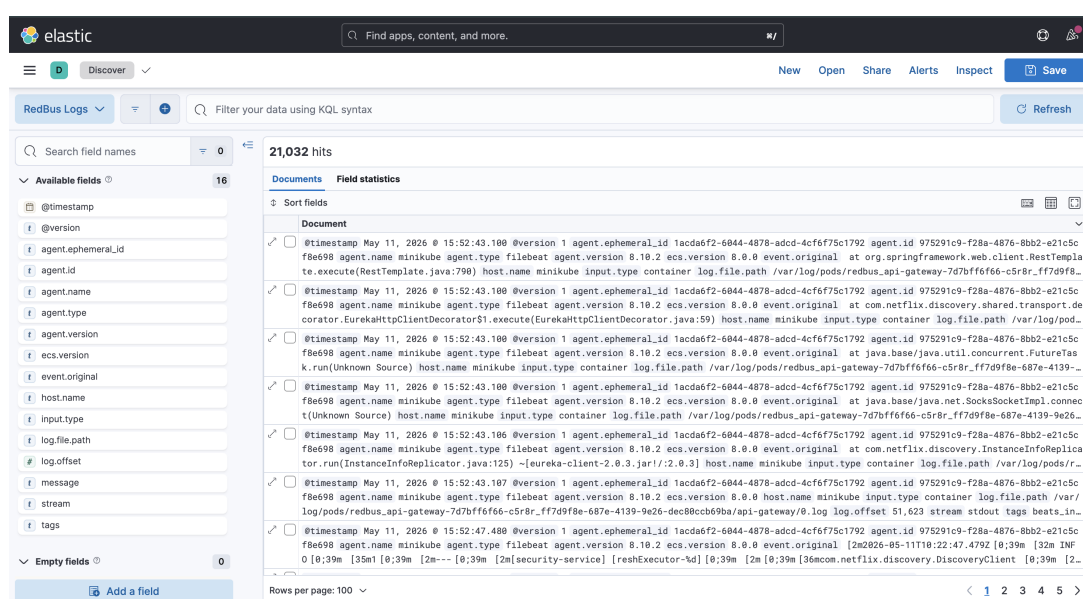


Figure 11: ELK Stack Deployment in Kubernetes

Explanation: This image shows the ELK stack deployment architecture in Kubernetes. The stack consists of Elasticsearch for centralized log storage and indexing, Logstash for log collection and processing, and Kibana for visualization and querying. Filebeat DaemonSets run on all Kubernetes nodes to collect pod logs from `/var/log/pods` and forward them to Logstash. This distributed log collection ensures complete coverage of all microservices and infrastructure components. The metadata enrichment from Kubernetes allows operators to filter and correlate logs by namespace, pod name, container, and service, providing rich observability for distributed debugging and incident response.

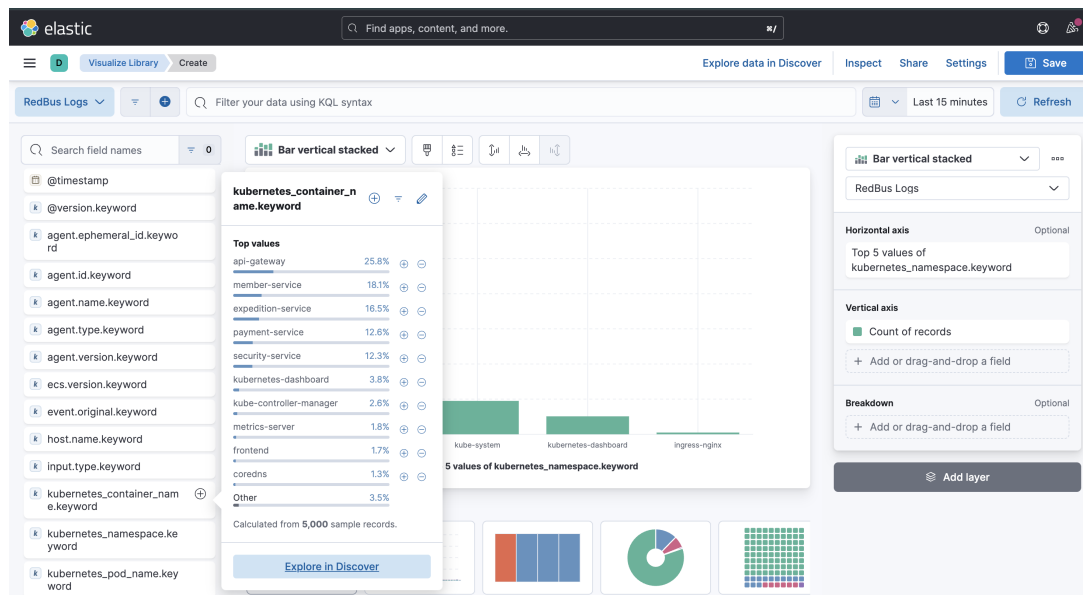


Figure 12: Elasticsearch Indexing and Storage

Explanation: Elasticsearch serves as the search and analytics engine for the centralized logging infrastructure. It receives parsed log documents from Logstash and indexes them for rapid full-text search across millions of log entries. This figure demonstrates the storage layer that enables operators to query logs by service name, error type, timestamp ranges, and custom fields. The indexing strategy allows historical log retention while supporting real-time search for troubleshooting production issues. Kubernetes metadata enrichment ensures every log entry includes context about which service, pod, and node it originated from, enabling cross-service correlation and trace analysis.

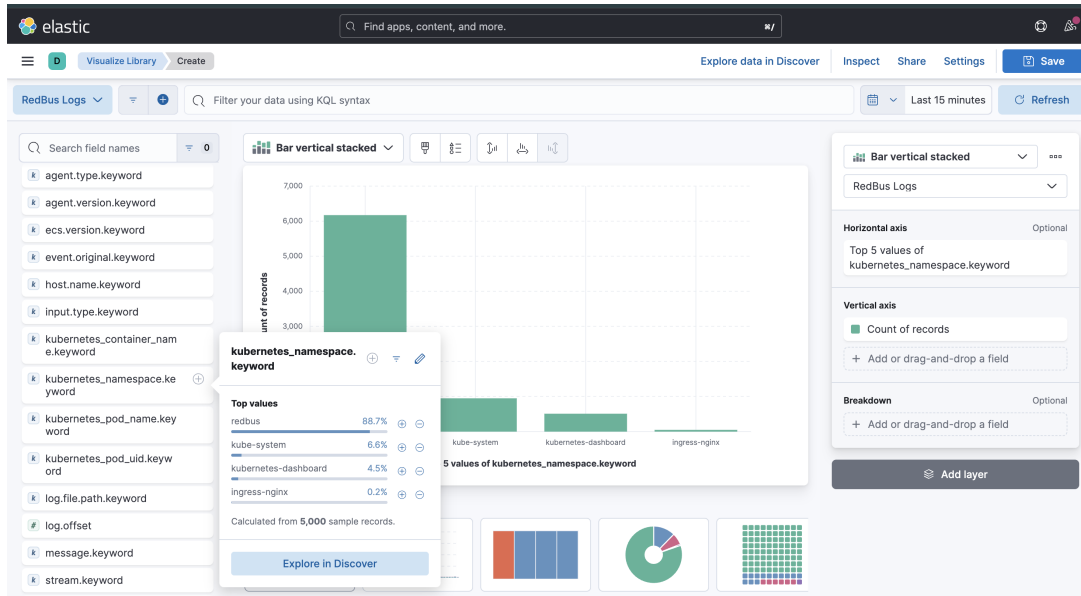


Figure 13: Kibana Dashboards and Log Visualization

Explanation: Kibana provides the user-facing interface for the ELK stack, exposed through Kubernetes Ingress at the `/kibana` endpoint. Operators use Kibana dashboards to visualize log patterns, create alerts, and investigate anomalies. The dashboard capabilities include filtering by service type, time ranges, error codes, and custom fields extracted during Logstash parsing. Kibana’s ability to create visualizations of log volume, error rates, and performance metrics helps identify bottlenecks and degraded services. This observability tool is essential for the DevOps workflow because it enables rapid incident response by providing historical context and real-time insights into distributed system behavior across all microservices.

11 Results, Evaluation, and Conclusion

11.1 Evaluation Alignment

The implementation satisfies the expected SPE deliverables:

- Git-based version control and structured repository organization.
- Automated CI/CD pipeline for build, image generation, and deployment.
- Containerized services and Kubernetes deployment manifests.
- Centralized logging and visualization via ELK.
- Advanced operational features such as HPA and IaC-style automation with Ansible.

11.2 Conclusion

RedBus demonstrates that a domain-focused application can be built together with a practical production delivery model. The project combines software design, deployment automation, and runtime observability into one integrated workflow, which is the core objective of software production engineering.

The system is suitable for final-project evaluation as both a functional platform and an operations-aware engineering artifact.

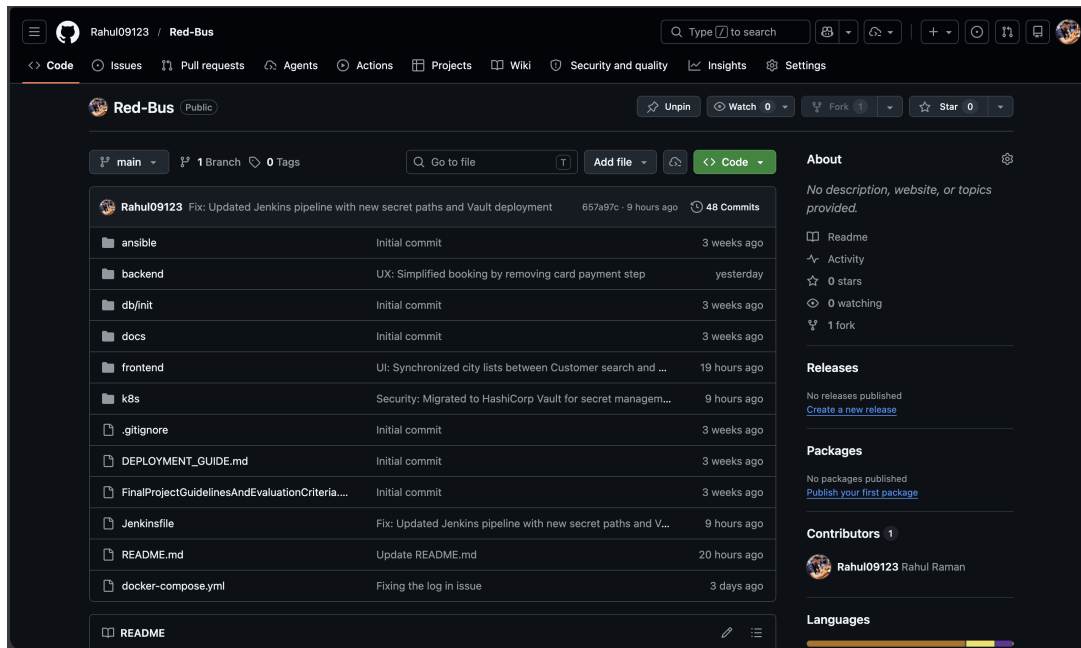


Figure 14: GitHub Repository - Complete Source Code

Explanation: The complete implementation and source code for the RedBus Ticketing System is available on GitHub at <https://github.com/Rahul09123/Red-Bus>. This repository contains all microservices, Kubernetes manifests, Jenkins pipeline configurations, Ansible playbooks, database initialization scripts, and infrastructure-as-code components. The repository demonstrates production-ready practices including branching strategy, commit history traceability, CI/CD integration, and comprehensive documentation. All deployment configurations, security policies, and observability setup are version-controlled and can be replicated for testing or deployment to different environments.